

Laboratory Work No. 1

Training Classifiers using PyTorch

Student Name: [Your Name]

Group/ID: [Your Group]

Course: IPIS-2026

Variant No.: [e.g., 3]

Dataset: [e.g., CIFAR-10]

Instructor: Associate Prof. Aliaksandr Kroshchanka

Date: June 23, 2026

Brest State Technical University
Department of Intelligent Information Systems

Abstract

This report details the implementation and evaluation of a Convolutional Neural Network (CNN) built with the PyTorch framework. The primary objective is to construct a neural network classifier and train it on a standard computer vision dataset from `torchvision.datasets`. The report covers model architecture design, training optimization using a specific optimizer, evaluation of testing accuracy, and comparison with state-of-the-art results.

Contents

1	Introduction and Objectives	2
2	Variant Details	2
3	Model Architecture	2
4	Training Process and Results	3
4.1	Loss Curve	3
4.2	Evaluation	3
5	State-of-the-Art Comparison	3
6	Visualization of Model Inference	3
7	Conclusion	4
A	Assignment Variants	5

1 Introduction and Objectives

Laboratory Goal

To learn how to construct neural network classifiers and perform their training on well-known computer vision datasets.

The specific tasks to be completed in this laboratory work are:

1. Construct a Convolutional Neural Network (CNN) model and train it on the assigned dataset.
2. Evaluate the training efficiency on the test set and plot the error curve (loss) over epochs.
3. Review state-of-the-art results for the dataset and draw conclusions regarding the trained model's performance.
4. Implement a visualization module that takes an arbitrary image, feeds it to the architecture, and outputs the predicted class.

2 Variant Details

According to the assignment, the model is built and trained according to the parameters specified in Table 1.

Table 1: Task Assignment Configuration

Parameter	Value
Variant Number	[Insert Variant, e.g., 3]
Dataset	[e.g., CIFAR-10]
Image Size	[e.g., 32x32]
Optimizer	[e.g., SGD]
Loss Function	CrossEntropyLoss

3 Model Architecture

Following the guidelines, preference was given to simple architectures utilizing basic layer types: convolutional layers, fully connected (linear) layers, subsampling (pooling) layers, and non-linear activation functions (e.g., ReLU).

Below is a snippet of the implemented CNN using PyTorch:

```

1 import torch.nn as nn
2 import torch.nn.functional as F
3
4 class SimpleCNN(nn.Module):
5     def __init__(self, num_classes=10):
6         super(SimpleCNN, self).__init__()
7         # Convolutional layers

```

```
8     self.conv1 = nn.Conv2d(in_channels=3, out_channels=16,
9                             kernel_size=3, padding=1)
10    self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
11    self.conv2 = nn.Conv2d(in_channels=16, out_channels=32,
12                            kernel_size=3, padding=1)
13
14    # Fully connected layers
15    self.fc1 = nn.Linear(32 * 8 * 8, 128) # Adjust dimensions based
16    on input size
17    self.fc2 = nn.Linear(128, num_classes)
18
19    def forward(self, x):
20        x = self.pool(F.relu(self.conv1(x)))
21        x = self.pool(F.relu(self.conv2(x)))
22        x = x.view(-1, 32 * 8 * 8) # Flatten the tensor
23        x = F.relu(self.fc1(x))
24        x = self.fc2(x)
25        return x
```

Listing 1: PyTorch Model Definition

4 Training Process and Results

The model was trained using the `torchvision.datasets` module. The recommended `CrossEntropyLoss` criterion was applied to compute the gradient, and the weights were updated using the assigned optimizer.

4.1 Loss Curve

Figure X (insert your matplotlib plot above) illustrates the reduction of training loss over time, confirming that the network successfully learned the underlying features of the dataset.

4.2 Evaluation

The final accuracy achieved on the test set was **XX.X%**.

5 State-of-the-Art Comparison

Current state-of-the-art (SOTA) models for the assigned dataset (e.g., Vision Transformers or Deep ResNets) typically achieve accuracies upwards of 99%. The simple CNN implemented in this laboratory work achieved a lower accuracy, which is expected due to the shallow architecture, limited number of parameters, and the absence of advanced regularization or data augmentation techniques. However, the model successfully demonstrated the fundamental principles of deep learning.

6 Visualization of Model Inference

To verify the practical capability of the network, an arbitrary image was selected from the test set and passed through the model.

The model successfully extracted features and assigned the correct class label with high confidence, demonstrating an understanding of the visual representation.

7 Conclusion

In this laboratory work, a Convolutional Neural Network was successfully designed, implemented, and trained from scratch using PyTorch. The basic layers (convolutional, pooling, and fully connected) proved sufficient for learning fundamental features of the provided dataset. Future improvements could include the implementation of deeper networks, dropout layers, and hyperparameter tuning to bridge the gap toward state-of-the-art results. The full source code can be found in the attached GitHub repository.

References

- [1] PyTorch Foundation. (2023). *PyTorch Documentation*. Retrieved from <https://pytorch.org/docs/>
- [2] Koehler, G. *Nextjournal PyTorch MNIST Guide*. Retrieved from <https://nextjournal.com/gkoehler/pytorch-mnist>

A Assignment Variants

Each student is assigned a specific dataset and optimizer based on their variant number. The configuration for all 20 variants is presented in Table 2.

Table 2: Complete List of Assignment Variants

Variant No.	Dataset	Image Size	Optimizer
1	MNIST	28×28	SGD
2	Fashion-MNIST	28×28	SGD
3	CIFAR-10	32×32	SGD
4	CIFAR-100	32×32	SGD
5	STL-10 (labeled)	96×96	SGD
6	MNIST	28×28	Adam
7	Fashion-MNIST	28×28	Adam
8	CIFAR-10	32×32	Adam
9	CIFAR-100	32×32	Adam
10	STL-10 (labeled)	96×96	Adam
11	MNIST	28×28	Adadelta
12	Fashion-MNIST	28×28	Adadelta
13	CIFAR-10	32×32	Adadelta
14	CIFAR-100	32×32	Adadelta
15	STL-10 (labeled)	96×96	Adadelta
16	MNIST	28×28	RMSprop
17	Fashion-MNIST	28×28	RMSprop
18	CIFAR-10	32×32	RMSprop
19	CIFAR-100	32×32	RMSprop
20	STL-10 (labeled)	96×96	RMSprop